

HTTP Methods Simplified

Beginner Tutorial for College Joiners



BEGINNER FRIENDLY

</> WEB DEVELOPMENT

The Basics

What is HTTP?

HTTP = Communication Protocol

HTTP stands for **HyperText Transfer Protocol** — the foundation of how data is exchanged between a client and a server on the web.

It works as a simple cycle: **Request** → **Response**

Every time you open a website, your browser sends an HTTP request and the server sends back a response. [Learn more at blog.postman.com](https://blog.postman.com)

👉 Simple Example

Think of it like ordering at a restaurant:

- **Client (Browser)** — places the order (request)
- **Server** — prepares and delivers the food (response)

 Every website interaction you've ever had is powered by HTTP requests and responses.

Understanding the Purpose

Why HTTP Methods?

HTTP Methods are like **verbs** — they tell the server exactly **what action to perform** on a resource. Without them, the server wouldn't know whether you want to read, create, update, or delete data.



They Are Verbs (Actions)

Just like verbs describe actions in a sentence, HTTP methods describe what action should be taken on the server's data.



Used in APIs

Every REST API you'll ever build or consume relies on HTTP methods to define what operation is being requested.



Used in Web Apps

From login forms to profile updates, every web application action maps to an HTTP method behind the scenes.

The Foundation

CRUD Mapping

Every application performs four fundamental operations on data, known as **CRUD**. Each maps directly to one or more HTTP methods.

Create POST Add new data to the server		Read GET Retrieve existing data
Update PUT / PATCH Modify existing data		Delete DELETE Remove data from server

[Read more about REST API & CRUD at `geeksforgeeks.org`](https://www.geeksforgeeks.org/rest-api-crud/)

At a Glance

HTTP Methods Overview

There are five core HTTP methods every beginner must know. Each serves a distinct purpose in communicating with a server.

Method	Purpose	Description
GET	Read data	Retrieve information from the server without modifying it
POST	Create data	Send new data to the server to create a resource
PUT	Replace data	Fully replace an existing resource with new data
PATCH	Update partially	Update only specific fields of an existing resource
DELETE	Remove data	Delete a resource from the server permanently

GET METHOD

GET = Read Data

The **GET** method is used to **fetch data from the server**. It is the most commonly used HTTP method and is completely safe — it does **NOT** **modify anything** on the server.

Think of it as simply *looking* at something without touching it.

[Learn more at restfulapi.net](https://restfulapi.net)

Example Request

```
GET /users
```

What it returns

- A list of all users
- No data is changed on the server
- Safe to call multiple times



👉 Real life: Opening a website in your browser sends a GET request to load the page.

POST METHOD

POST = Create Data

The **POST** method is used to **create a new resource** on the server. Unlike GET, POST sends data in the **request body**, making it suitable for submitting forms and creating records.

Every time you sign up for a new account, a POST request is sent behind the scenes.

[Learn more at restfulapi.net](https://restfulapi.net)

Example Request

POST /users

```
{  
  "name": "Alice",  
  "email": "alice@example.com"  
}
```

What it does

- Creates a new user record
- Data is sent in the request body
- Server returns the created resource



👉 Real life: Filling in and submitting a sign-up form.

PUT METHOD

PUT = Full Update

The PUT method **replaces an entire resource** with new data. You must send the **complete data** — any fields you leave out will be removed or reset.

Use PUT when you want to fully overwrite a record with fresh information.

[Learn more at geeksforgeeks.org](https://www.geeksforgeeks.org/put-http-method/)

Example Request

PUT /users/123

```
{  
  "name": "Alice Smith",  
  "email": "alice@example.com",  
  "age": 22  
}
```

What it does

- Replaces the entire user record
- All fields must be included
- Missing fields may be lost



👉 Real life: Updating your full profile — name, email, age, and all.

PATCH METHOD

PATCH = Partial Update

The **PATCH** method updates **only the specific fields** you provide. Unlike PUT, you don't need to send the entire resource — just the fields you want to change.

This makes PATCH more efficient when only a small change is needed.

[Learn more at geeksforgeeks.org](https://www.geeksforgeeks.org/)

Example Request

```
PATCH /users/123
```

```
{  
  "email": "newemail@example.com"  
}
```

What it does

- Updates only the email field
- All other fields remain unchanged
- More efficient than PUT for small changes



👉 Real life: Changing only your email address in account settings.

DELETE METHOD

DELETE = Remove Data

The **DELETE** method is used to **permanently remove a resource** from the server. Once deleted, the resource is gone — the server will typically return a success status with no content.

Use with caution — deletions are usually irreversible.

[Learn more at geeksforgeeks.org](https://www.geeksforgeeks.org/delete-http-method/)

Example Request

```
DELETE /users/123
```

What it does

- Removes user with ID 123
- No request body needed
- Server returns 200 OK or 204 No Content



👉 Real life: Clicking "Delete Account" in your app settings.

Side-by-Side

Quick Comparison

Here's a concise reference table to help you remember what each HTTP method does at a glance.



GET

Fetch existing data from the server.
Read-only, safe to repeat.



POST

Create a brand new resource. Data goes
in the request body.



PUT

Replace an entire resource. All fields
must be provided.



PATCH

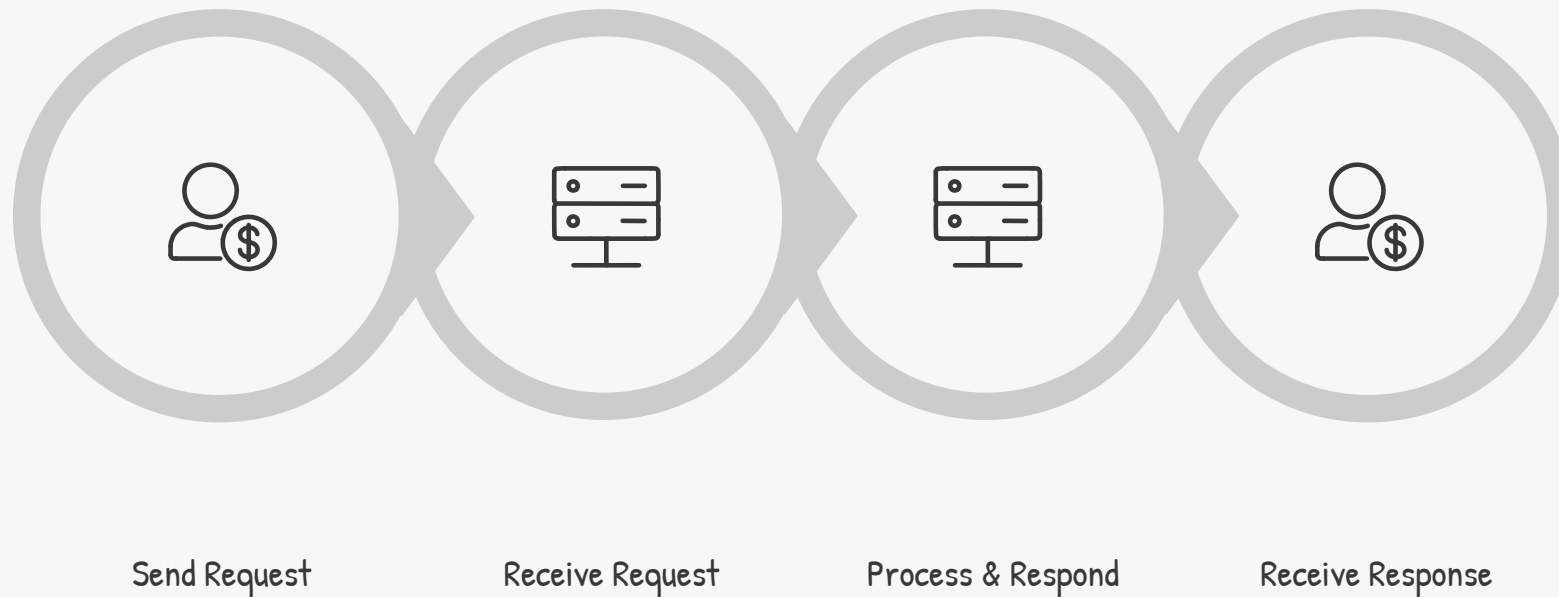
Modify only selected fields. Efficient for small updates.



DELETE

Remove a resource permanently from the server.

Request → Response Flow



Every HTTP interaction follows this fundamental cycle. The client (your browser or app) sends a request using one of the HTTP methods, the server processes it, and returns an appropriate response — such as data for GET or a confirmation for POST.

GET Example

Client sends `GET /users` → Server returns a list of users as JSON data.

POST Example

Client sends `POST /users` with body → Server creates the user and returns the new record.

Putting It Together

Real-Life Mapping

Every everyday action in a web application maps to a specific HTTP method. Here's how common user actions translate to the methods you've learned.

Real-Life Action	Method	What Happens
View your profile page	GET	Server fetches and returns your profile data
Register a new user account	POST	Server creates a new user record in the database
Update your full profile	PUT	Server replaces your entire profile with new data
Change only your password	PATCH	Server updates only the password field
Delete your account	DELETE	Server permanently removes your account



Notice how each action has a clear, single method — this is the beauty of RESTful API design.

Practice Time

Hands-on Demo with Postman

Postman is a free tool that lets you send HTTP requests to any API without writing code — perfect for beginners to experiment and learn.

01

Download & Open Postman

Go to **postman.com**, download the app, and create a free account to get started.

03

Try a POST Request

Switch to POST, add a JSON body with a name and email, and send. Watch the server create a new user record.

02

Try a GET Request

Enter **GET https://jsonplaceholder.typicode.com/users** and hit Send. You'll see a list of users returned as JSON.

04

Try a DELETE Request

Use **DELETE /users/1** and observe the server's response confirming the resource has been removed.



Use the free public API at **jsonplaceholder.typicode.com** — it's perfect for practising HTTP methods safely.

Quick Quiz

Let's see how much you've learned! Match each scenario to the correct HTTP method. Think carefully before answering.

1

Question 1

You want to **fetch data** from the server. Which HTTP method do you use?

2

Question 2

You want to **create a new user** in the database. Which method do you use?

3

Question 3

You want to **update a user's full profile** — name, email, age, and all fields. Which method?

4

Question 4

You want to **update only the email** address of an existing user. Which method is most appropriate?

5

Question 5

You want to **permanently delete a user** from the system. Which method do you use?



Answers: 1 → GET | 2 → POST | 3 → PUT | 4 → PATCH | 5 → DELETE

Wrapping Up

Summary

You've now covered all five core HTTP methods. These are the building blocks of every API and web application you'll ever work with.

HTTP Methods = Actions

They tell the server **what to do** — read, create, replace, update, or delete a resource.

5 Methods Are Enough

GET, POST, PUT, PATCH, and DELETE cover everything a beginner needs to build and consume APIs.

Foundation for APIs

Mastering these methods is your first step into backend development, REST APIs, and full-stack engineering.

Keep practising with Postman, explore real APIs, and you'll be building your own in no time. **Happy coding!** 🎉